Técnico especialista em tecnologias e programação de sistemas de informação

citeforma

Relatório Projeto Markos

UC 00618: Criar aplicações em linguagem de programação Python

Formando: Márcio Santos
Formador: João Galamba
Lisboa, Março de 2026

Introdução e Objetivos

Markdown é uma linguagem de formatação cujo o objetivo é formatar textos de forma simples, rápida e legível. Através do uso de marcadores, este permite converter linhas de texto em elementos HTML que podem assim ser utilizados para expor o texto formatado em diversas plataformas.

Alguns exemplos de utilização de Markdown inclui, como por exemplo, documentação em projetos de programação, onde a plataforma GitHub utiliza um README.md, ".md" sendo o formato Markdown, para assim demonstrar uma breve descrição de um projeto na página inicial do projeto. Outro exemplo são plataformas de comunicação ou chats, como o Discord, que incluem funcionalidades de Markdown que permitem ao utilizador decorar as suas mensagens ao enviar, por vezes até demonstrando o resultado antes de enviar. E ainda mais um exemplo inclui a capacidade de ser usado em qualquer editor de texto, pois os mesmo costumam incluir uma funcionalidade Markdown embutida, que permite assim ao utilizador copiar e colar textos de um editor para o outro sem perder a sua formatação.

O projeto "Markos" tem como objetivo replicar esse comportamento, onde dado um ficheiro .md ou um texto com marcadores Markdown, permite devolver o texto formatado com elementos HTML que pode ser devolvido ao utilizador ou em forma de texto, ou num ficheiro HTML pronto a usar e perfeitamente funcional.

Desenho e Estrutura

Estrutura de pastas

O projeto Markos tem a seguinte organização de pastas:
“”“

```
Markos/
├──.venv/
├──.vscode/
├──docs/
├──src/
└──tests/
```

“”“

Componentes Principais

E o funcionamento do programa depende dos seguintes componentes:
“”“

```
src/
├──html_backend.py
├──latex_backend.py
├──markdown_backend.py
├──markdown_compiler.py
├──markdown_list.py
├──markos.py
└──utils.py
```

“”“

Onde:

- html_backend: Implementação específica para a geração de tags HTML
- latex_backend: Implementação específica que gera instruções LaTeX
- markdown_backend: Define a classe abstrata que permite criar os backends
- markdown_compiler: Responsável por ler e determinar os tokens markdown
- markdown_list: Prepara os conteúdos de uma lista markdown
- markos: Base do programa que recebe o ficheiro markdown e devolve o HTML
- utils: Contém funções de suporte

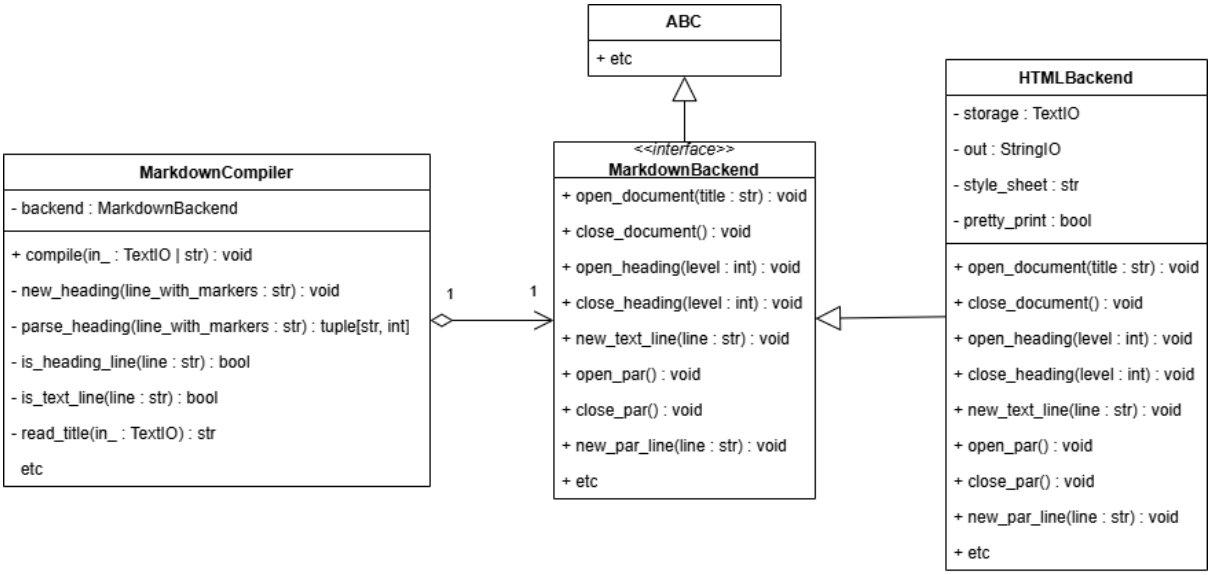
Parte 1 (Videos)

A parte 1 define a fase do projeto de visualização de vídeos onde foram criadas a base do projeto, a interpretação de parágrafos e heading tal como a interpretação de listas markdown. Esta fase do projeto demonstrou a importância de um ambiente virtual em projeto, tal como introduziu conceitos como implementar uma máquina de estados e utilizar expressões regulares para classificar linhas do ficheiro. Também introduziu a biblioteca docopt para permitir criar um programa que funciona em linhas de comando como os parâmetros personalizados. Por fim foi introduzida a biblioteca beautifulsoup4, utilizada ao fechar o programa para indentar o texto html e devolver um resultado mais legível para o utilizador.

Diagramas (Parte 1)

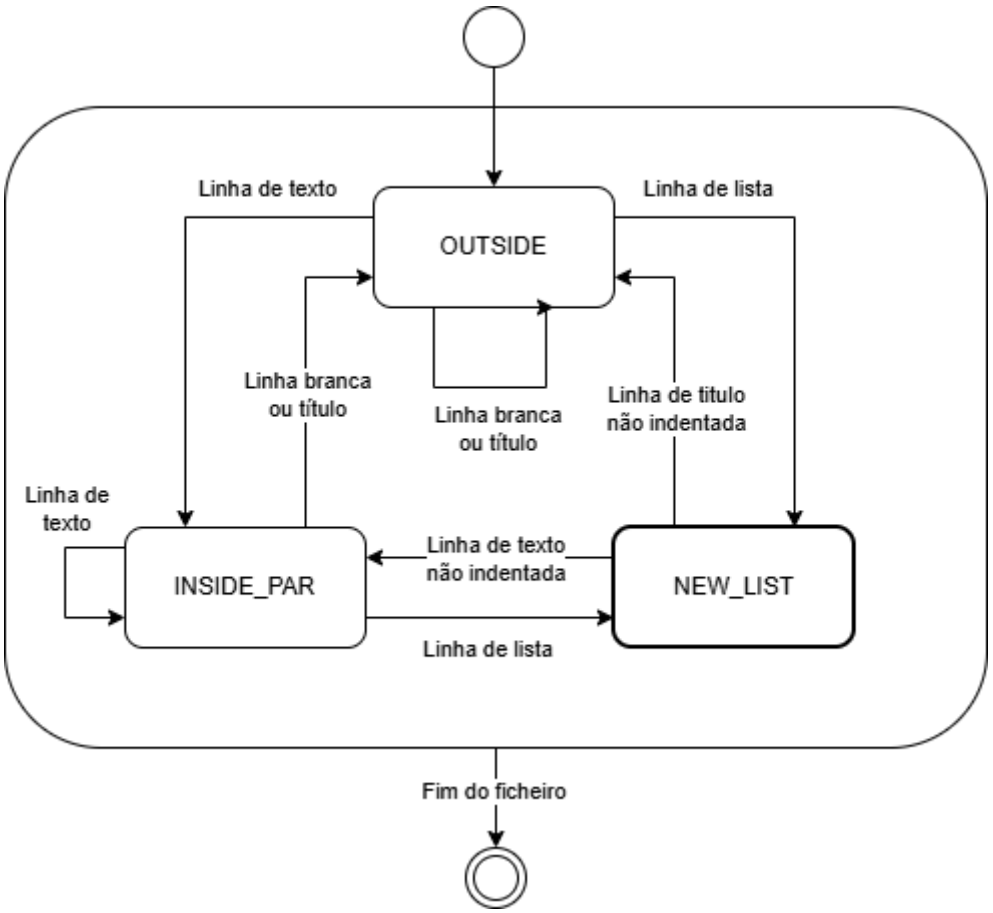
Backend e Compilador

Este diagrama de classes ilustra a arquitetura modular do sistema, destacando a agregação entre o MarkdownCompiler e a interface MarkdownBackend. Esta estrutura permite que o compilador seja independente do formato de saída, facilitando a extensibilidade do software.



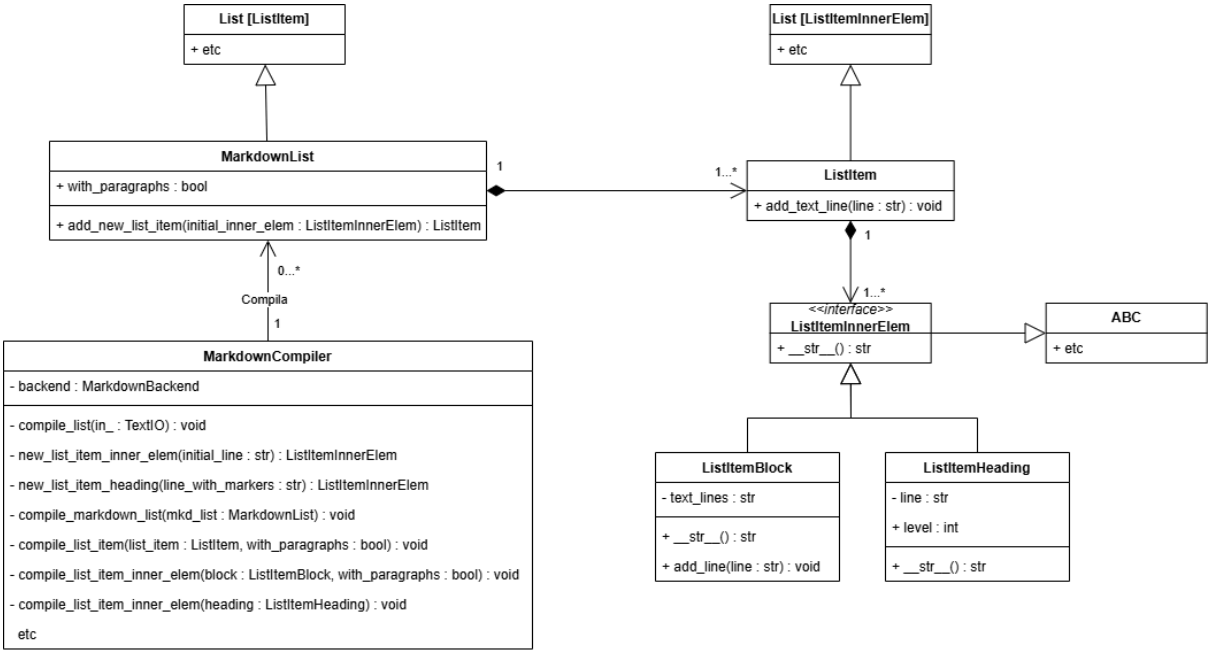
Máquina de Estados Principal

Aqui temos o diagrama de estados baseado na mesma apresentada nos vídeos da parte 1, mas construída à minha maneira. Este diagrama apresenta 3 estados: OUTSIDE, INSIDE_PAR (parágrafo) e NEW_LIST tal como a forma que estas transitam de estados. O estado NEW_LIST em particular está destacado pois apresenta uma lógica de estados internos.



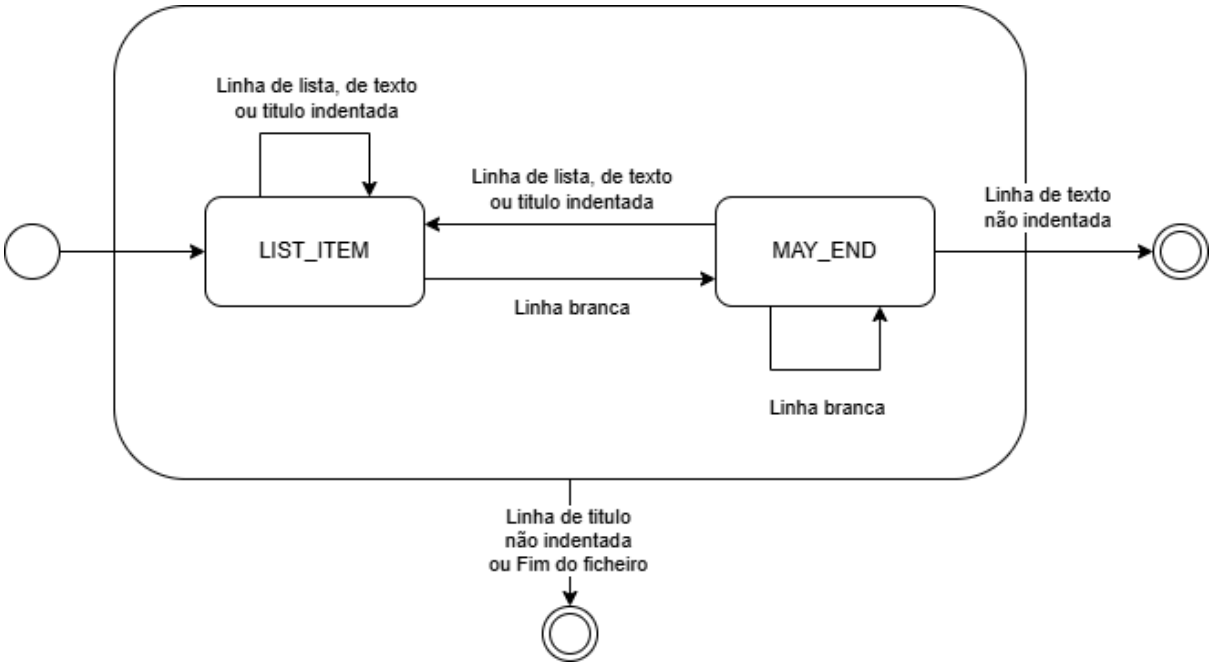
Classes de Listas

Apresenta a hierarquia das classes de suporte à manipulação de listas, evidenciando a especialização a partir das classes nativas do Python e a sua interação com o processador central durante a deteção de blocos de listagem.



Máquina de Estados de Lista

Este diagrama, também baseado num diagrama apresentado nos vídeos, representa a lógica de estados internos de NEW_LIST da máquina de estados principal. Neste caso temos 2 estados presentes, sendo LIST_ITEM e MAY_END, tal como as transições internas inclusive a saída que volta à máquina de estados principal.



Parte 2 (Restantes Funcionalidades)

Marcador de Listas '*':

O marcador '*' para reconhecimento de listas não-ordenadas foi adicionado ao programa alterando a variável que define o marcador de '-' para '[-*]' pois segundo as regras de expressões regulares, os parênteses retos são usados para identificar 1 destes caracteres.

Listas Numeradas:

Para implementar listas numeradas, foi criado um marcador para identificar as notações '1.' e '1', que foi definido da seguinte forma: '\d+\.?', e a seguir criado um identificador de expressão regular com a mesma estrutura do que foi usado para as listas não-numeradas. Tendo o identificador e marcador criados, foram depois alteradas algumas funções relativamente ao compilador da lista para incluir um parâmetro booleano que identifique se a lista é não numerada, assim pude simplesmente adicionar a verificação da expressão regular na máquina de estados principal enquanto a máquina de estados de lista só tinha de se preocupar com distinguir as duas.
Por fim, atualizei o backend para gerar o HTML de acordo com esse parâmetro booleano, e fui garantir que as duas listas na máquina de estados de lista se mantinham separadas obrigando ao fecho da lista caso um marcador da outra lista fosse detectado.

Elementos Inline (Negrito e Itálico):

Para implementar os elementos inline negrito e itálico, foram criadas quatro expressões regulares, padrões distintos para cada marcador onde '*' e '_' são lidados de forma diferente de modo a impedir o '_' criar itálicos para nomes de ficheiro que contenham underscore pelo meio.
Depois foi criado um método apply_style() no compilador que recebendo a linha, o padrão regex e as funções de abertura e fecho dos elementos, irá realizar um .split() na linha a partir do regex e filtrar o texto dentro dos marcadores e adicionar o tag html nesse texto a partir das funções fornecidas, devolvendo no final uma string da linha com tags embutidos.
Desta forma foi adicionado mais quatro métodos no backend para abrir e fechar ambos negrito e itálico. Foram também criados os métodos handle_bold() e handle_italic() que recebem a linha e aplicam o método apply_style() a com ambas as respectivas expressões regulares de '*' e '_'.
E por fim, para juntar tudo, foi criado o método compile_inline() que recebe o texto da linha e chama ambos os métodos handle_bold() e handle_italic() que por sua vez é chamado somente quando o compilador principal ou o compilador de listas tenta adicionar as linhas ao ficheiro de saída.

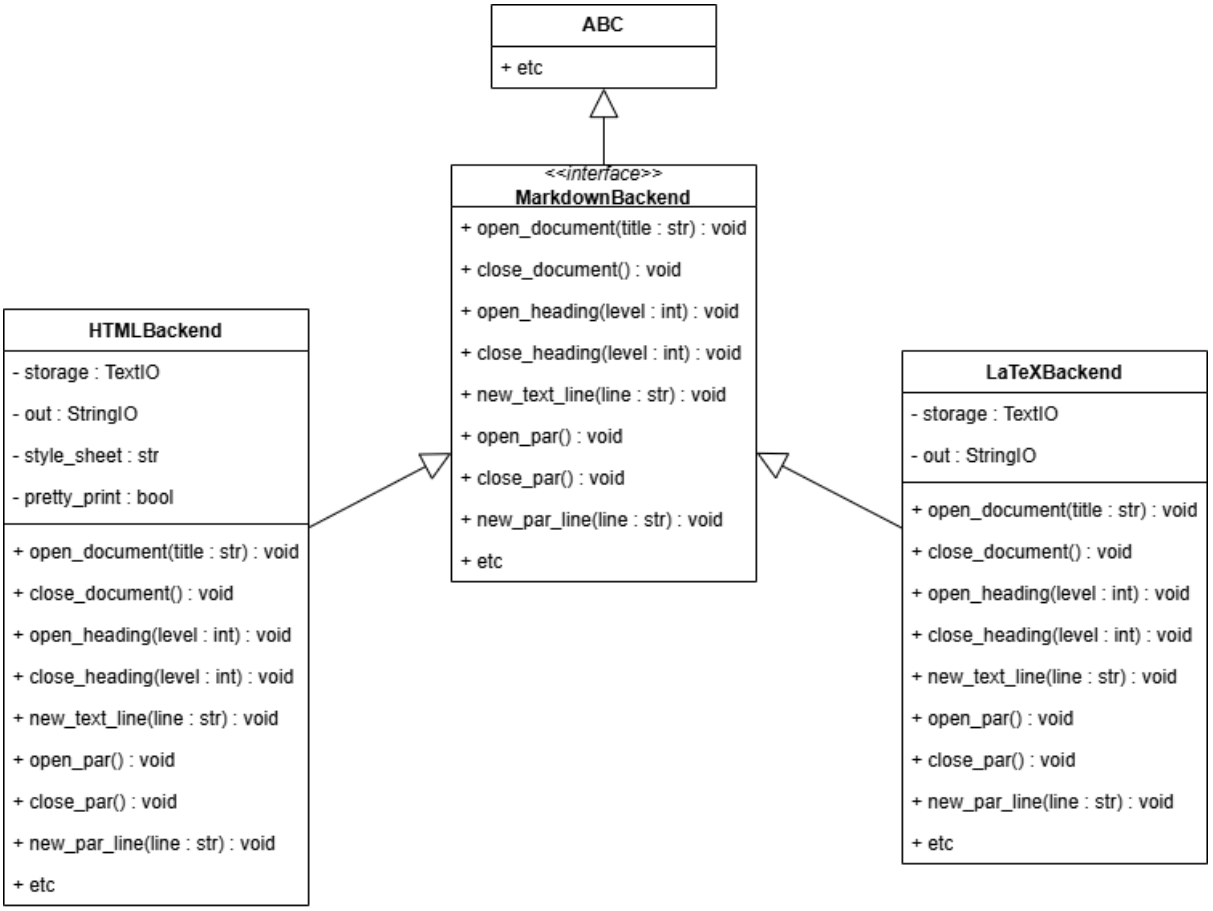
Elementos Inline (Ligações e Imagens):

Para implementação dos elementos inline de ligações e imagens, foi utilizado um truque similar aos negritos e itálicos, onde foram criadas duas expressões regulares que pudessem captar três grupos de captura.
A seguir foi criado o método apply_media(), que tem uma função similar ao apply_style() dos outros elementos de estilo, mas adaptado para lidar com três grupos de captura onde é feita a sanitização, e como o terceiro grupo de captura é opcional, garantir que converte None a uma string vazia antes de aplicar o respetivo método do backend.
Para o backend foi-se optado por criar um método apenas para cada um dos elementos html, denominados assim de new_link() e new_image(). E depois para poder ligar ao compile_inline() dos elementos de estilo, foi criado um método handle_media() que realiza a verificação e troca de texto, começando pelas imagens devido ao marcador extra '!'.

Diagramas (Parte 2)

Relação entre Backends

Este diagrama de classes ilustra a implementação do polimorfismo no sistema Markos. Através da herança da classe abstrata MarkdownBackend, foram criadas duas implementações concretas: HtmlBackend e LatexBackend.
Enquanto partilham a mesma interface de métodos (assegurando que o compilador possa alternar entre formatos de saída de forma transparente), as classes distinguem-se nos seus atributos internos e lógica de formatação. O HtmlBackend integra funcionalidades de estilização externa (CSS) e pós-processamento (Prettify), enquanto o LatexBackend foca na integridade sintática e no tratamento de caracteres reservados, demonstrando a versatilidade da arquitetura modular adotada.



Implementação

Bibliotecas e Versões usadas

Para a realização deste projeto foi utilizado um ambiente virtual (venv) de modo a garantir que as versões das bibliotecas e Python sejam constantes independente da máquina onde seja executado. Este programa utiliza a versão Python 3.14.2, e contém a biblioteca docopt, na versão 0.6.2, para permitir controlar os parâmetros de compilação por terminal, e utiliza também a biblioteca beautifulsoup4 de forma a permitir ao utilizador gerar, opcionalmente, o HTML de output indentado.

Como Executar

Para executar, tem-se de garantir que o ambiente virtual está ativo, isso pode ser feito de várias formas dependendo de sistema operativo e terminal. Como este programa foi escrito em Windows, vou mencionar os métodos para powershell e cmd, onde partindo da raiz:

- Powershell: “.\.venv\Scripts\Activate.ps1”.
- cmd: “.venv\Scripts\activate”.

Com o ambiente virtual ativo, a execução deve começar da pasta “src/” e seguir a seguinte estrutura:

- python Markos.py [-s STYLE_SHEET] [-p] [-l] [INPUT_FILE] [OUTPUT_FILE]

Alternativamente pode ser usado no lugar de “python Markos.py”:

- Powershell: Markos.ps1
- cmd: Markos.cmd

Docopt Docstring

Já a interface docopt foi definida através da seguinte docstring:

```
"""
  Usage:
    {sys.argv[0]} [-s STYLE_SHEET] [-p] [-l] [INPUT_FILE] [OUTPUT_FILE]

  Options:
  INPUT_FILE        Markdown source file. Default: stdin
  OUTPUT_FILE       Output source file. Default: stdout
  -h, --help        This help
  -p, --pretty      Prettify HTML output
  -l, --latex       Switch output format to LaTeX (default: HTML)
  -s STYLE_SHEET, --style-sheet=STYLE_SHEET   Use this STYLE_SHEET
"""
```

Elementos Extra Implementados

Backend LaTeX:

Na fase final do projeto, foi implementada uma nova classe de backend dedicada à compilação de Markdown para LaTeX. A integração foi realizada através da extensão da interface de linha de comando via docopt, permitindo ao utilizador ativar este modo através da flag -l ou --latex. No ponto de entrada do programa (markos.py), esta opção determina qual a instância de backend que será utilizada pelo compilador.
A classe LatexBackend herda da classe abstrata MarkdownBackend, reimplementando todos os métodos para a sintaxe correspondente. Dada a natureza do LaTeX, os atributos de Pretty Print e Style Sheet foram omitidos nesta implementação. Adicionalmente, foi desenvolvido um sistema de escape de caracteres sensíveis, garantindo que símbolos reservados da linguagem LaTeX (como %, & ou $) sejam devidamente processados para evitar erros de compilação no documento final.

Conclusão

O projeto Markos resultou num conversor funcional de Markdown para HTML, que permite identificar diversos elementos como secções, parágrafo e lista, e apresentá-los num ficheiro HTML devidamente estruturado, com a opção de adicionar estilização a partir de um ficheiro exterior .css e até apresentar a estrutura de forma legível a partir do beautifulsoup4.

Durante a realização do projeto, foram encontrados alguns problemas, maioritariamente envolvendo uma função que volta linhas atrás que estavam relacionados à inconsistência de EOL (end of line) e cálculo de offset para retroceder num ficheiro relacionado à codificação inconsistente do mesmo.

Durante o desenvolvimento do backend LaTeX, surgiu um desafio técnico crítico relacionado com o processamento de sequências de escape, especificamente a barra inversa (\). Uma vez que este carácter é o delimitador de comandos no LaTeX, a sua presença no texto original do Markdown causava erros de compilação imediatos.
Para mitigar este problema, foi necessária a implementação de uma lógica de sanitização de strings adicional. Este novo método no backend interseta e trata qualquer ocorrência de barras inversas antes da escrita final (e não só), assegurando que o texto literal seja distinguido dos comandos do sistema, garantindo assim a estabilidade do documento gerado.

Foi implementado:

- Indentação (Prettify)
- Importação de ficheiros CSS
- Título de Página
- Parágrafos
- Capítulos e Secções (Headings)
- Listas não numeradas começadas por ambos '-' e '*'
- Listas numeradas começadas por ambos '1.' e '1'
- Elementos em linha Itálico e Negrito
- Elementos em linha Ligações e Imagens
- Backend LaTeX